

Technical and theory explanations — Assessment

Task 2

Grimdark Forge · COSC3046 · Augustus Zibell-Barnes (S4246254)

These notes explain, module by module and concern by concern, *how* the prototype meets the AT2 criteria and *why* it is built that way. File and function names are the real ones in the repository so a marker can open the file beside the explanation.

1. Request lifecycle (applies to every page)

1. `require_once shared/bootstrap.php` — starts the session, defines `GF_ROOT`, loads `store.php`, `validation.php`, `i18n.php`, `images.php` and the helper functions.
2. The page reads its inputs with `filter_input()` and, for a POST, checks `csrf_ok()`.
3. The page fetches or mutates the store through `collection('name')`, which returns a *reference* into the single array loaded by `gf_store()`.
4. The page sets `$page_title` / `$page_description`, requires `shared/header.php`, prints its `<main>` content, and requires `shared/footer.php`.
5. At shutdown `gf_store_flush()` writes the array back (APCu or scratch file) **only if its serialised hash changed**, then releases the lock.

Because step 3 hands back a reference, module code writes to the arrays directly (`$posts[$id]['body'] = ...`) and never has to call a "save" function — the flush is automatic.

2. The in-memory store (`shared/store.php`)

Problem. The brief requires hard-coded associative arrays and no database, yet a post made by one visitor must be visible to the next. PHP's process model destroys all variables at the end of each request.

Solution. The seed arrays from `shared/data.php` are loaded once, then parked between requests:

Backend	When	Persistence	Concurrency
APCu (<code>apcu_fetch</code> / <code>apcu_store</code>)	extension present and enabled	shared memory, cleared on web-server restart	atomic per key
Scratch file <code>sys_get_temp_dir()/grimdark-forge-<md5 of root>.store</code>	APCu absent	serialised <code>serialize()</code> blob, outside the web root	<code>flock(LOCK_EX)</code> held from first read to shutdown

Design points:

- The key includes `md5(GF_ROOT)`, so two copies of the site on one server (e.g. two students' home directories) never share data.
- `unserialize($raw, ['allowed_classes' => false])` — only arrays and scalars can come back, so a corrupted or tampered file cannot instantiate objects.
- The exclusive lock is taken *before* reading and released *after* writing, so two simultaneous replies cannot overwrite each other (a classic lost-update bug).
- Read-only requests do not write: the flush compares `md5(serialize($store))` with the hash captured at load time.
- `reset_demo_data()` calls `gf_store_replace(gf_seed_data())` — the same code path as first load, so "reset" and "fresh server" are guaranteed identical.

Per-visitor state (username, locale, guest cart, CSRF token, flash messages) stays in `$_SESSION`. Everything shared between visitors is in the store. That split is the whole reason two markers in different browsers can interact.

3. Security handling

3.1 Input filtering

Every request parameter is read with `filter_input()` rather than the superglobals:

```
$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT);           // int or false/null
$raw = filter_input(INPUT_POST, $name, FILTER_DEFAULT, FILTER_REQUIRE_SCALAR);
```

`FILTER_VALIDATE_INT` rejects `?id=1 OR 1=1`-style input outright; `FILTER_REQUIRE_SCALAR` rejects `name[]=...` array injection into string fields. `form_values()` in `validation.php` does this for every declared field, trims it, and coerces to string, so module code never sees `null` or arrays.

3.2 Cross-site request forgery

- `csrf_token()` — a 32-hex-character token from `random_bytes(16)`, created once per session.
- `csrf_field()` — a hidden `<input name="csrf">` printed inside every `<form method="post">` (forum, blog, reviews, cart, checkout, account, locale toggle, reset).
- `csrf_ok()` — `hash_equals()` (constant-time) between the session token and the posted one. Every POST handler calls it before any store mutation; a failure flashes "Your session has expired. Please try again." and redirects back to the form without acting.

Why it matters: without it, a hostile page could submit `delete-post.php` on behalf of a logged-in visitor. The token is unguessable and never leaves the site, so a forged request cannot carry it.

3.3 Output escaping (XSS)

`e($value)` wraps `htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')`; `te()` translates then escapes. Every dynamic value printed into HTML — titles, bodies, usernames, query-string echoes, even the `alt`

text a member typed — goes through it. `paragraphs()` escapes first and only then converts blank lines into `<p>` elements, so user text can never introduce markup.

3.4 Ownership and authorisation

`owns($author) is current_user() === $author`. It is used twice:

1. In templates, to decide whether to render Edit/Delete controls.
2. In `edit-*.php` and `delete-*.php`, **before** any work: a failing check returns `http_response_code(403)` (or 404 if the record does not exist) with an explanatory page.

Hiding a button is not security; step 2 is. This is why typing `forum/edit-post.php?id=1` as the wrong user gives a real 403 rather than a form.

3.5 Payment data

`type => 'card'` strips spaces, requires 13–19 digits and passes `luhn_ok()`; expiry must be `MM/YY` in the future; CSC is 3–4 digits. The order record stores **only** `card_last4 = substr($digits, -4)`; the full number, expiry and CSC are discarded after validation and never enter the store or the session.

3.6 File uploads

`resolve_image()` → `validation.php` checks size ($\leq 1\,048\,576$ bytes) and then `getimagesize()` on the temp file, accepting only `image/jpeg|png|gif|webp` as *detected*, not as the browser claimed. The bytes are base64-encoded into a `data:` URI inside the store — nothing is ever written under the web root, so a malicious upload cannot become an executable file on the server.

3.7 Open-redirect prevention

`safe_return($candidate, $default)` only accepts a relative, same-site path (no scheme, no leading `//`), so the `?return=` parameter used by login and the locale toggle cannot be pointed at an external site.

4. Validation twice (`assets/js/validate.js` + `shared/validation.php`)

Single source of truth. Each form declares a `$rules` array in PHP:

```
'review-rating' => ['label' => 'Rating', 'required' => true, 'type' => 'int', 'int_min' => 1, 'int_max' => 5],
```

The same array is printed onto the control as `data-required data-type="int" data-min="1" data-max="5"`. `validate.js` reads those attributes on submit; `validate()` in PHP applies them again on the server. Supported rule types: `required`, `min/max length`, `type` (`int`, `email`, `date`, `card`, `expiry`, `csc`, `postcode`, `phone`, `username`), `int_min/int_max`, `in` (allowed values), `match` (regex), `same` (confirm field), `checked` (checkbox), plus `required-if-file` for alt text.

No HTML validation attributes. `required`, `pattern`, `min`, `max`, `maxlength` and `type="number"` are absent from every template; the scripted pre-commit check greps for them. This is deliberate: browser-native validation cannot be translated, styled or announced consistently, and it would mask whether the PHP layer actually works.

Identical rendering. Both layers write into the same DOM: the summary `<div id="form-errors" role="alert" tabindex="-1">` receives focus and lists every message as an in-page link; each failed control gets `aria-invalid="true"` and its `aria-describedby` extended to point at `#<name>-error`. Disable JavaScript and the page looks and behaves the same after a round-trip.

Server wins. JavaScript is a convenience; if it is disabled, bypassed or out of date, the PHP result is the one that decides whether the store changes.

5. Module CRUD operations

All four modules share the same page shape: a **list** page with filter/sort form, a **detail** page, a **new** page, an **edit** page and a **delete** confirmation page. Each `lib.php` holds the data-access functions; templates only read.

5.1 Discussion Forum (`forum/`)

Operation	Page	Store change
Create	<code>new-thread.php</code>	inserts a row in <code>threads</code> and an opening post in <code>posts</code> (<code>parent_id => null</code>); replies from <code>thread.php</code> insert a <code>posts</code> row with <code>parent_id</code>
Read	<code>boards.php</code> (<code>threads_query()</code> — filter by title, text, author, board; sort by activity, created, title, reply count; paginated 6/page), <code>thread.php</code> (<code>thread_posts()</code> builds the tree via <code>post_children()</code>)	none
Update	<code>edit-post.php</code>	edits title, body, image; for an opening post also the thread title and board; records an optional edit reason
Delete	<code>delete-post.php</code>	soft: <code>deleted => true</code> . The row stays in memory for audit; <code>post_children()</code> re-attaches orphaned replies to the nearest visible ancestor

5.2 Shopping Cart (shop/)

Operation	Page	Store change
Create	<code>product.php</code> → <code>cart_add()</code>	adds a line (product, variant, quantity, note) to the user's cart, keyed by username (guest carts keyed by session id and merged on login via <code>gf_merge_guest_cart()</code>)
Read	<code>catalogue.php</code> (filter by title, category, colour, size, price, stock; sort; 8/page), <code>cart.php</code> (sort/filter lines), <code>order-confirmation.php</code> (only the owning account)	none
Update	<code>cart.php</code>	change variant, quantity or note per line; apply/remove <code>FORGE10</code>
Delete	<code>cart.php</code>	<code>cart_remove()</code> a line; <code>cart_clear()</code> after checkout
Checkout	<code>checkout.php</code>	validates delivery and card fields, re-checks stock, decrements <code>products[*]['stock']</code> , writes an <code>orders</code> row with totals in cents and <code>card_last4</code>

`cart_totals()` is the only place money is computed: subtotal → percentage discount → delivery (free over \$120 standard) → GST = total ÷ 11 → grand total, all integer cents.

5.3 Blog (blog/)

Operation	Page	Store change
Create	<code>new-post.php</code> (shared <code>form.php</code>)	inserts <code>posts</code> row: title, author, date, tags, summary, body, image; comments inserted from <code>post.php</code>
Read	<code>posts.php</code> (<code>blog_query()</code> — search by title, text, author, tag, date range; 6/page), <code>post.php</code> (full post + comments, <code>reading_minutes()</code>)	none
Update	<code>edit-post.php</code> ; comment edit inline on <code>post.php</code>	replaces the fields of the owner's row
Delete	<code>delete-post.php</code>	hard: <code>unset()</code> the post and every comment on it (the confirmation page states the comment count)

5.4 Product Review & Rating (reviews/)

Operation	Page	Store change
Create	new-review.php (shared form.php)	inserts reviews row: product, title, description, rating 1–5, reviewer, date, image
Read	reviews.php (reviews_query() — filter by product, title, text, minimum rating, reviewer, date range; reviews_breakdown() star histogram; 6/page), review.php	none
Update	edit-review.php	replaces fields of the owner's row
Delete	delete-review.php	hard: unset()

Product averages are not stored: `product_rating()` in `shop/lib.php` computes them from the live `reviews` collection on every request, so the shop's star display can never drift from the reviews module.

6. Locale mechanism (shared/i18n.php , shared/locale.php , shared/lang/de-DE.php)

- **Selection:** a `<form method="post">` in the header, so it works without JavaScript; `site.js` upgrades it into a disclosure menu (`aria-expanded` , `aria-controls` , Escape, outside click, focus moved into the list).
 - **Storage:** `$_SESSION['locale']` , whitelisted against `shared/i18n.php` .
 - **Strings:** `t('Log in')` looks the English key up in the active dictionary; a miss returns the English, never a blank. The dictionary has 747 entries. `sprintf` -style `%1$s` placeholders keep word order translatable.
 - **Formatting:** `money()` , `number()` , `fmt_date()` use ICU (`NumberFormatter` , `IntlDateFormatter`) when `intl` is loaded, otherwise a two-locale manual table. The output is wrapped (`money_html()` , `time_html()` with `<time datetime>`), and `site.js` re-formats the same nodes with `Intl.NumberFormat` / `Intl.DateTimeFormat` so the browser's own locale data is used when available.
 - **Declaration:** `<html lang="<?= locale() ?>">` so screen readers switch pronunciation.
-

7. Accessibility compliance

Requirement	How it is met
Landmarks	Real <code><header></code> , <code><nav aria-label></code> , <code><main id="main"></code> , <code><footer></code> on every page from the shell; skip link is the first focusable element
Headings	Exactly one <code><h1></code> in <code><main></code> ; no skipped levels (checked by script on all 23 × 2 pages)
Forms	<code><label for></code> on every control; hints linked with <code>aria-describedby</code> ; errors add <code>aria-invalid</code> and extend <code>aria-describedby</code> ; error summary <code>role="alert"</code> receives focus
Images	<code>alt</code> on every <code></code> ; member-attached images <i>require</i> a text alternative (validated); all decorative SVGs are text-free
Keyboard	Locale menu, cart controls and all forms operable by keyboard; focus ring defined once in <code>base.css</code> and never removed
Colour	Rating and stock status carry text as well as colour (<code>rating_html()</code> , <code>stock_status()</code>)
Language	<code><html lang></code> follows the active locale; all <code>aria-label</code> / <code>alt</code> strings are translatable sentences
Motion / JS	Everything works with JavaScript disabled; JS only enhances

8. Known limitations (declared, not hidden)

1. The scratch-file fallback for the store is a file, though never queried as a database.
2. Uploaded images as data URIs (≤ 1 MB) enlarge the store; library images are preferred.
3. Soft-deleted forum posts have no moderator view yet.
4. Reply re-attachment after a soft delete can read oddly; the delete page warns.
5. Pagination controls only appear once content exceeds a page.
6. W3C validator and Lighthouse have not yet been run against the hosted copy.